University of Deusto

Bachelor in Data Science and Artificial Intelligence

INTEGRATION OF VISION LANGUAGE MODELS (VLMS) IN A MULTI-AGENT REINFORCEMENT LEARNING (MARL) ENVIRONMENT FOR COOPERATIVE ROBOTIC TASKS IN NVIDIA ISAAC SIM

Student Unai Olaizola Osa

DNI 73032586L

Email unai.olaizola.osa@opendeusto.es

GitHub repository VLMs IsaacSim

April 17, 2026

Acknowledgments

Abstract

This project proposes the development and implementation of a Multi-Agent Reinforcement Learning (MARL) system integrated with Vision Language Models (VLMs) for the execution of cooperative robotic tasks within a simulated NVIDIA Isaac Sim environment.

One of the major challenges lies in translating commands from a natural language format into precise robotic actions. To address this, the reasoning capabilities of the VLMs have been exploited for complex instruction interpretation and decision-making. A controlled environment where robotic agents must cooperate to manipulate and respond to objects in order to achieve a shared goal has been designed and shaped for Reinforcement Learning training using NVIDIA's Isaac Lab framework. The integration of these models has enhanced generalization and promote a more cooperative behavior of the agents against variations in the scene. The learning efficiency has been evaluated in addition to metrics such as the stability of the policies and the transferable capacity of the model(s). Additionally, the project includes an in-depth analysis of state-of-the-art (SOTA) models, providing a comprehensive review of the current methodologies and core concepts that form the theoretical foundation of the proposed architecture, presented in the documentation.

The expected outcome is a demonstration and reflection of the empowerment of multi-modal models within robotic environments, which can be applied to real-world problems.

Index terms

Vision language models, multi-agent reinforcement learning system, computer vision

Contents

List of Figures

List of Pseudocodes

1. INTRODUCTION

1.1 PROBLEM CONTEXT

1.2 MOTIVATION

1.3 MAIN OBJECTIVES AND SPECIFIC OBJECTIVES

1.4 PROJECT SCOPE AND LIMITATIONS

2. THEORETICAL FOUNDATIONS

2.1 VISION LANGUAGE MODELS

Vision language models (VLMs) are artificial intelligence (AI) models that combine both computer vision (CV) and natural language processing (NLP) techniques and capabilities, these models are designed to jointly understand and reason visual and textual information simultaneously [1] [3]. Among the most well-known models which will be covered when corresponded, models such as GPT-5, Gemini and LLaVA could be highlighted.

In comparison to the traditional models that process data modalities independently, VLMs have been shaped to align textual and visual data, enabling to reason complex scenes and natural language instructions.

2.1.1 History of VLMs

VLMs are the result of the evolution to earlier image captioning systems, systems which were designed primarily to take isolated images and produce descriptions. However, this systems just received the image as input, not an image-text pair input as the vision language models do now [6].

2.1.1.1 Earliest models

The early image captioning systems from around year 2010 used the *encoder-decoder* architecture, which will be described later on. These models would encode the patched images and vectorize them into feature vectors, which were then fed to the decoder part in order to generate the associated output description. When it comes to the strategies implemented by this earliest models include combination of handcrafted *visual features* to encode the image patches and *n-gram* templates to generate the description.

Those visual features can refer to any visual property like points, edges or even objects in the image.

On the other hand, n-grams are statistical language models that calculate the probability of the next word in a sequence from a fixed size window of previous words. Depending on the number of previous words considered, we can get bigram models if just one previous word is considered, if it's two is a trigram model, ... up to n - 1 considered as *n-gram* models and finally an unigram if no previous context is considered.

The general approximation for a sequence of words $w_1, \ldots, w_m$ is:

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-(n-1)}, \ldots, w_{i-1})$$

- Unigram Model ($n = 1$): No context, independent probabilities.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i)$$

- Bigram Model ($n = 2$): Depends on the single previous word.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-1})$$

- Trigram Model ($n = 3$): Depends on the two previous words.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-2}, w_{i-1})$$

2.1.1.2 Deep learning based models

When the deep learning neural networks arose and became dominant near year 2015, newer models and strategies also emerged. Convolutional neural networks (CNNs) started to being used for image encoding and recurrent neural networks (RNNs) to generate the captions.

However by 2018, the transformer architecture replaced the sequential recurrent neural networks in the role as decoders. In parallel, the network parameter training started relying on image-text pair datasets, and the scope of VLMs widened until reaching other tasks. Among those tasks, *phrase grounding* could be mentioned. Task involving both natural language processing and computer vision where words or phrases within a sentence are associated to corresponding patches in an image [4]. Consists of identifying the spatial location within an image that aligns with the meaning of a given phrase, enabling models to map visual and textual data.

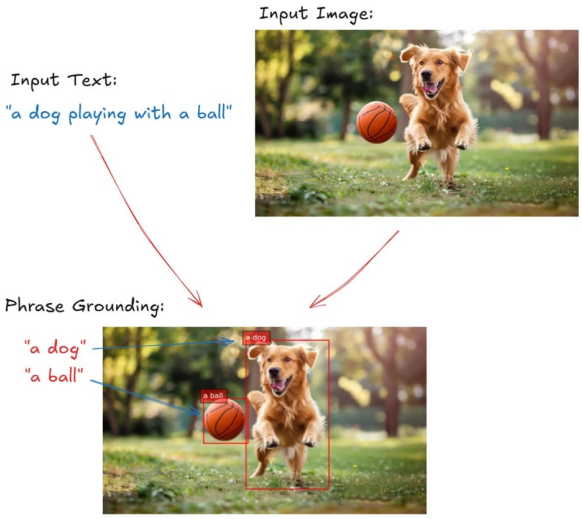

Figure 2.1: Graphical example of phrase grounding

Another task which still remains as perhaps the most relevant one is *visual question answering* (VQA). Specific task where the model is provided with an image and a specific related question, and it must generate an accurate response to it based on the visual content. Being its core pillars the object detection duty for identifying the objects in the image, then scene understanding to map the relations between objects and then finally text comprehension for embedded text interpretation within an image [2].

2.1.1.3 Advancements in 2020s - CLIP

In the year 2021, OpenAI launched the approach that would totally revolution the evolution of vision language models, they released the method of *contrastive language-image pretraining* also known as *CLIP*. Technique for training an image-text pair into neural network models, having an exclusive neural network for image and text understanding purposes, adopting a contrastive objective.

When it comes to the algorithm itself, each neural network receives the input in the corresponding format and returns a single semantic/visual representative vector [5]. The objective to maximize by both models is the best alignment between both output vectors in the shared vector space, mapping the similar text-image pairs as close as possible while putting apart the most dissimilar ones. The training relies on a huge dataset with image-text pairs as and the two output vectors are considered similar depending on the *dot product* value.

$$s_{i,j} = \mathbf{v}_i \cdot \mathbf{w}_j = \sum_{k=1}^{d} v_{i,k} w_{j,k} \tag{2.1}$$

where $\mathbf{v}_i$ and $\mathbf{w}_j$ represent the embedding vectors generated by the text and image encoders respectively, and d denotes the dimensionality of the vector space. The loss function used is the multi-class N-pair loss, which is a symmetric *cross-entropy loss* over the similarity scores. The function fosters the dot product between the matching image and text vectors to be as high as possible, while discouraging high dot products between mismatching pairs.

$$\mathcal{L} = -\frac{1}{N} \sum_{i} \ln \frac{e^{v_i \cdot w_i / \tau}}{\sum_j e^{v_i \cdot w_j / \tau}} - \frac{1}{N} \sum_{j} \ln \frac{e^{v_j \cdot w_j / \tau}}{\sum_i e^{v_i \cdot w_j / \tau}} \tag{2.2}$$

being parameter T the temperature parameter ($T>0$) which is parametrized and learned during the training. Other loss functions are possible, such as the *Sigmoid CLIP* function.

2.1.2 Elements of a VLMs

Inspired in other previous model architectures, vision language models consisted on two main components: the *language encoder* and the *visual encoder*. Compared to other *seq2seq* architectures like encoder-only, decoder-only or encoder-decoder, VLMs consist on 2 encoders, one for each data modality encoding task.

2.1.2.1 Transformers

Before going in detail of the components, I will go over the basic architecture most of the VLMs are based on: *transformers*.

1. The encoders transform the input sequence into numerical representations called *embeddings* which capture all the semantic and positional meaning of the tokens belonging to the input.

2. Then they use a mechanism called *self-attention* which allows the encoder to focus on the most relevant tokens of the sequence independent of their position.

3. Finally the decoders of the transformers take both the output of the self-attention mechanism and the encoder generated embeddings to generate the most statistically likely output sequence.

2.1.2.2 Language encoder

This first encoder captures the semantic relations and meanings between the words in the textual prompt given and turns them into *text embeddings* to be processed, carrying the same encoding process seen in the majority of architectures.

2.1.2.3 Vision encoder

The second encoder extracts the visual properties and patterns from the visual data of the prompt and transforms them into embeddings too. Compared to earlier VLMs which used *convolutional neural networks* (CNNs) for feature extraction such as the colors, shapes and textures from the input, modern ones employ an architecture called *visual transformer* (ViT).

2.1.2.4 Visual transformer (ViT)

Visual transformers process images into smaller patches and treats them as sequences, implementing also the self-attention mechanism across the patches to create a transformer-based representation of the input message.

2.2 REINFORCEMENT LEARNING

2.3 MULTI-AGENT SYSTEMS

2.4 STATE OF THE ART

2.4.1 VLMs evolution

2.4.2 Current models

2.4.3 Benchmarks

2.4.4 VLMs in robotics

2.4.5 Frameworks

3. SOLUTION APPROACH

3.1 SOLUTION PROPOSAL

3.2 SYSTEM ARCHITECTURE

3.3 TOOLS USED

3.4 WORKFLOW

4. SOLUTION DEVELOPMENT

4.1 ISAACSIM ENVIRONMENT DEFINITION

4.1.1 Agents

4.1.2 Cameras

4.1.3 Physics

4.2 INDIVIDUAL COMPONENT OPERATION

5. EXPERIMENTATION AND EVOLUTION

5.1 FIRST SCENARIOS

5.2 LAST SCENARIO

5.3 SUCCESSFUL AND FAILURE TRIES

5.4 RESULTS OBTAINED IN EACH SCENARIO

5.5 RESULT DISCUSSION

6. CONCLUSIONS

6.1 CONCLUSIONS AND LESSONS LEARNED

6.2 LIMITATIONS FOUND

The main drawbacks I have found during the development and research process during the whole work have been mainly hardware-related as the specific software requirements necessary to carry on the development are severely demanding due to the fact that the majority of the processes are run and rendered in the GPU. Although I have had access to a good graphical processing unit, the other components of the my working device have fallen behind. Taking for instance the minimum requirements of the used framework *IsaacSim*, I found myself in the limits of the bare minimum, and some of my components such as the CPU were worse than the required. However, I have been able to carry on anyways although there have been a few moments where the I have really felt the bottleneck. When it comes those moments I have felt limited and slowed downed due to my equipment are the following:

- I have needed to split some of the working components of the pipeline process inside *IsaacSim* as the whole process would not fit in my GPU's VRAM. The simple fact of opening the framework already consumed close to a quarter of the graphic card's total capacity, and starting the server to listen and interact with the VLM calls alongside left little to no room for more processes in the background. The fact of working on the edge of the minimum hardware requirements also slowed the rendering of visual work and reduced the number of frames per second, nevertheless, this did not obstruct the development process.

- When it comes to making the most of the reinforcement learning training process, I have needed to tweak some of the predefined system configurations to maximize the performance and reduce any kind of lagging and overflow. Adapting the GPU capacity properties to maximize the overall functioning. If I were to work with better equipment, I would have liked to go for longer and deeper trainings, however, I am happy with the performance I obtained when tweaking the parameters I will have discussed in the corresponding section and other strategies such as parallelization.

- Not all limitations have been hardware related, as the official NVIDIA documentation websites were usually corrupted depending on the version. Leading to consulting external pages to solve the mismatching dependencies or installation processes for the frameworks used.

- After consulting the official documentation of the *IsaacLab* repository, I found that even the default reinforcement learning training examples and demonstrations did not work as intended independently of the version. I was not the only one with the same issues as other peoples raised the same issues in different forums, so I inspired in the solutions given by the users with the same problems.

6.3 FUTURE WORK

BIBLIOGRAPHY

[1] Pietro Bolcato. Understanding vision-language models (vlms): A practical guide, 2024.

[2] Gravio Team. Vision-language models (VLM) vs visual question answering (VQA) in 2025? https://www.gravio.com/en-blog/vision-language-models-vlm-vs-visual-question-answering-vqa-in-2025, Mar 2025. Gravio Blog, Accessed: [Insert Date Here].

[3] IBM. What are vision-language models? https://www.ibm.com/think/topics/vision-language-models, 2024.

[4] M. Timothy. What is phrase grounding? https://blog.roboflow.com/what-is-phrase-grounding/, Nov 2024. Roboflow Blog, Accessed: [Insert Date Here].

[5] Wikipedia contributors. Contrastive language-image pre-training — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Contrastive_Language-Image_Pre-training, 2024. [Online; accessed 19-April-2026].

[6] Wikipedia contributors. Vision-language model — Wikipedia, the free encyclopedia, 2026.

7. APPENDIX

7.1 INSTALATION GUIDE

IsaacSim/Lab

7.2 RELEVANT CODE

7.3 PROMPTS USED

Prompts used for the VLM

7.4 RESULT TABLE

7.5 TENSORBOARD ANALYSIS